

Análise Experimental da Inferência SepConv em C++ com Ênfase em Eficiência Computacional e Uso de VRAM

Apresentação final, TCC2

Pedro Henrique Gomes

SENAC, Unidade Santo Amaro
Bacharelado em Ciência da Computação
Orientador: Prof. Afonso Brandão

São Paulo, 2026

Roteiro da apresentação

1. Introdução e problema
2. Objetivos e método
3. Desenvolvimento
4. Resultados e análise
5. Conclusão

Contexto: criar o quadro do meio

Quadro real no instante t | Quadro intermediário, $t + 0{,}5$ | Quadro real no instante $t + 1$



Fonte: elaborado pelo autor, com a sequência Beanbags do Middlebury (BAKER et al., 2011).

- Muitos vídeos têm 24 ou 30 FPS, e as telas atualizam de 60 Hz a 144 Hz.
- A interpolação de quadros (VFI) sintetiza as imagens que faltam entre dois quadros.

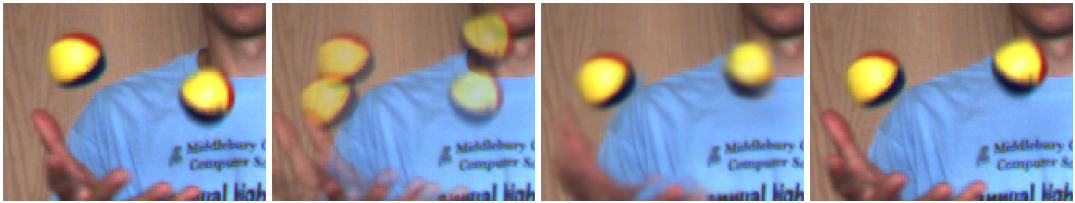
Repetir, misturar ou interpolar?

(a) Repetição
24,25 dB

(b) Mistura linear
26,64 dB

(c) SepConv++
31,88 dB

(d) Quadro real



Fonte: elaborado pelo autor, com a sequência Beanbags do Middlebury (BAKER et al., 2011).

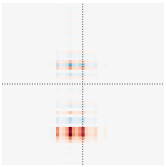
- **Repetição:** a bola fica na posição antiga.
- **Mistura:** surgem duas bolas semitransparentes (*ghosting*).
- **SepConv++:** uma bola só, perto da posição real, com algum borramento.

Filtros adaptativos: um filtro por pixel

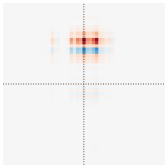
(a) Pixel na bola em movimento



Filtro no quadro t



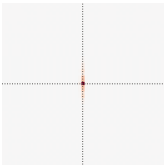
Filtro no quadro $t+1$



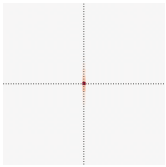
(b) Pixel em região parada



Filtro no quadro t



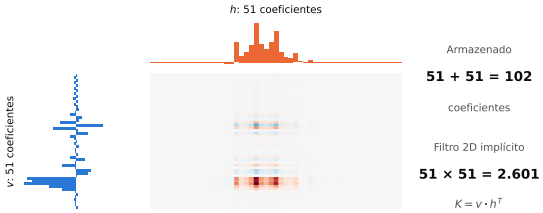
Filtro no quadro $t+1$

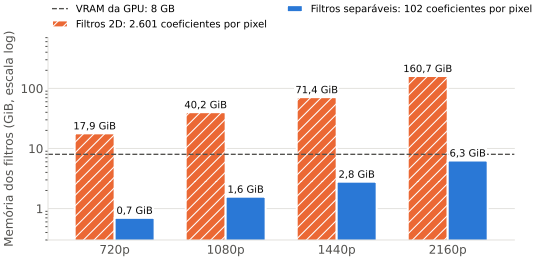


Fonte: elaborado pelo autor, com a sequência Beanbags (BAKER et al., 2011).

- A rede não gera o pixel: ela gera os pesos para sintetizá-lo.
- **Parado:** peso no centro do filtro.
- **Bola:** peso em $(-4, +16)$ no quadro t e em $(0, -14)$ em $t+1$.
- O movimento fica implícito nos pesos, sem fluxo óptico.

Separabilidade: menos memória nos filtros

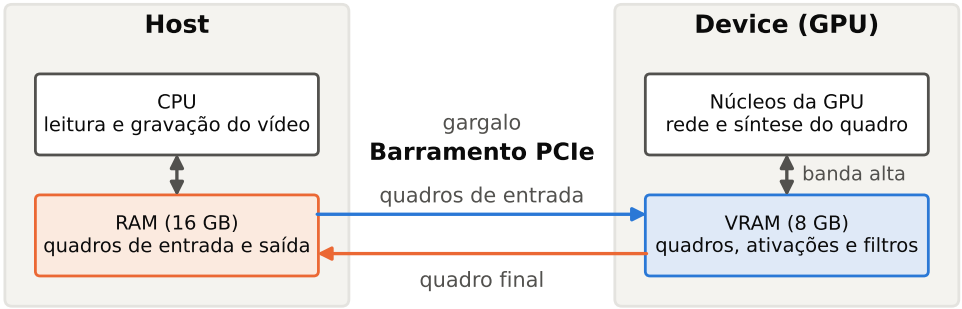




- Dois vetores 1D em vez de uma matriz 2D.
- 102 coeficientes por pixel, contra 2.601.
- A síntese continua com N^2 multiplicações: o ganho é de memória.

Em 1080p: **1,6 GB contra 40 GB**
É o que torna o modelo viável em 8 GB.

O gargalo: memória da GPU



- A VRAM é um limite físico: quando acaba, o *driver* usa a RAM e o tempo dispara.
- O barramento PCIe é mais lento que a memória interna da GPU.

Trabalhos correlatos

Trabalho	Estratégia	Limitação ou diferencial
Teed e Deng (2020)	RAFT: fluxo óptico recorrente, com atualização iterativa	Alto custo computacional e de VRAM
Niklaus et al. (2021)	SepConv++: convoluções adaptativas separáveis refinadas	Execução dependente do PyTorch
Han et al. (2024)	Movimento multiescala com pirâmide hierárquica	Maior complexidade da arquitetura
Este trabalho	**Inferência nativa: ONNX Runtime e *kernel* CUDA**	**Mede latência e VRAM em GPU de 8 GB**

O modelo é o mesmo do SepConv++: o que muda é como a inferência é executada.

Problema e pergunta de pesquisa

- Modelos de VFI dependem de *frameworks* de alto nível e exigem muita VRAM.
- Isso dificulta a execução em computadores com GPUs de consumo.

Pergunta de pesquisa

Em que medida um motor de inferência nativo em C++ e CUDA para o modelo SepConv reduz a latência e o consumo de VRAM em GPUs de consumo de 8 GB, mantendo a fidelidade visual da implementação original?

Objetivos

Objetivo geral

Determinar em que medida um motor nativo em C++ e CUDA reduz a latência e o pico de VRAM em uma GPU de 8 GB, em relação à implementação original, mantendo a fidelidade visual.

Objetivos específicos:

1. Verificar a equivalência numérica entre as saídas do motor e da original;
2. Quantificar a diferença de latência, de 720p a 2160p (H1);
3. Determinar o efeito sobre o pico de VRAM e onde a memória se esgota (H2);
4. Verificar a fidelidade no vídeo de teste e em conjuntos de referência (H3);
5. Identificar as fontes do consumo e medir alocador, precisão e blocos;
6. Separar o efeito da forma de execução do efeito da precisão.

Hipótese e critérios de validação

Um motor nativo reduz a latência e o pico de VRAM, sem degradar a fidelidade.

Critério	Condição, em cada resolução
H1: latência	Redução de pelo menos 10%, com teste t de Welch unilateral ($\alpha = 0{,}05$)
H2: VRAM	Pico de no máximo 90% do pico da original, sem saturar a GPU
H3: fidelidade	Diferença média de até 0,1 dB de PSNR, 0,001 de SSIM e 0,5 de VMAF

Os critérios valem para 720p e 1080p. As resoluções 1440p e 2160p mostram o que acontece quando a VRAM se esgota.

Método de pesquisa e cenário de testes

Estudo experimental comparativo

- **Variáveis manipuladas:** implementação, resolução e, depois, versão do motor e precisão.
- **Variáveis medidas:** tempo por quadro, pico de VRAM, PSNR, SSIM e VMAF.
- **Referência:** código original em Python/PyTorch.
- **Ambiente:** RTX 4060 Ti de 8 GB, i5-12400F, 16 GB de RAM, Windows 11.

Protocolo das baterias

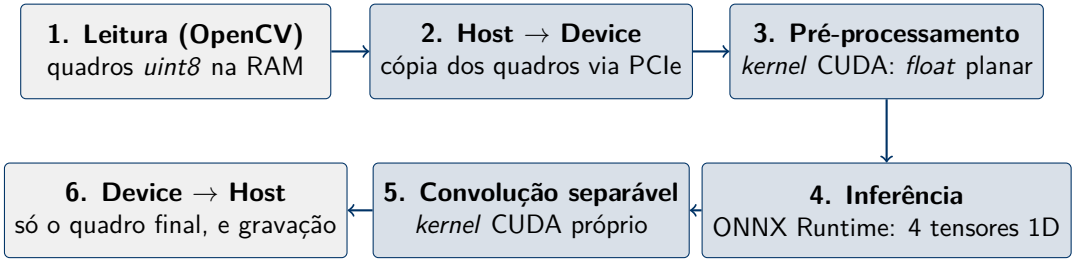
- Trecho de 61 quadros do trailer do GTA VI, em FFV1, de 720p a 2160p.
- Vídeo de 15 para 30 FPS: os quadros sintetizados são comparados aos reais.
- 27 medições após 3 de aquecimento (em 2160p, 3 após 1).
- VRAM pela NVML; a partir da 3ª bateria, a cada 1 ms.
- Qualidade também no Middlebury e no Vimeo-90K.

Solução desenvolvida: três versões

Versão	O que mudou	Pergunta respondida
v1	OpenCV, ONNX Runtime e *kernel* CUDA próprio, com o quadro inteiro	Só trocar a forma de execução já traz ganho?
v2	Processamento em blocos, com margem de 96 pixels	O pico de memória pode deixar de depender da resolução?
v3	Alocador ajustado e meia precisão (FP16)	De onde vinha o consumo excedente?

Cada versão nasceu do resultado da anterior e foi medida em uma bateria própria. Uma quarta bateria comparou as quatro combinações de implementação e precisão.

Arquitetura do motor



- Em azul, os estágios na VRAM: os dados só saem da GPU no quadro final.
- *I/O binding*: entradas e saídas do grafo apontam para *buffers* já alocados na VRAM.

Construção: exportação e síntese

Correção da exportação ONNX

- A primeira exportação só funcionava na resolução em que foi gerada.
- Uma condição em Python ficava fixa no grafo; foi trocada por um fatiamento válido para qualquer resolução com dimensões pares.

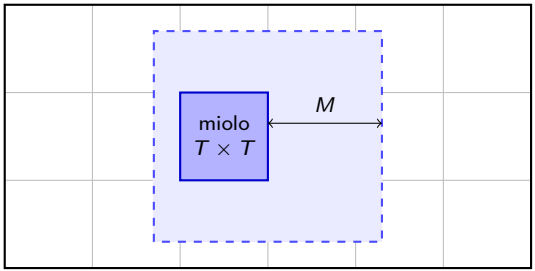
***Kernel* de síntese**

- Cada *thread* calcula um pixel, combinando os filtros 1D e os dois quadros.
- Os 51 coeficientes horizontais são lidos uma vez e reutilizados nas 51 linhas.

Equivalência com a original

93,67 dB de PSNR entre as saídas: 0,003% dos valores diferem, sempre em 1 nível.

v2: processamento em blocos



Tracejado: recorte enviado à rede (bloco + margem M). Só o miolo é aproveitado.

- Filtros e ativações passam a ter o tamanho do bloco.
- **Normalização global**, igual para todos os blocos.
- Recortes em **múltiplos de 16**, por causa da rede.
- **Margem de 96 pixels**: sem ela, erro de até 98 níveis; com ela, 1 nível em menos de 0,002% dos pixels.

v3: alocador e meia precisão

Alocador, em 1080p (quadro inteiro)

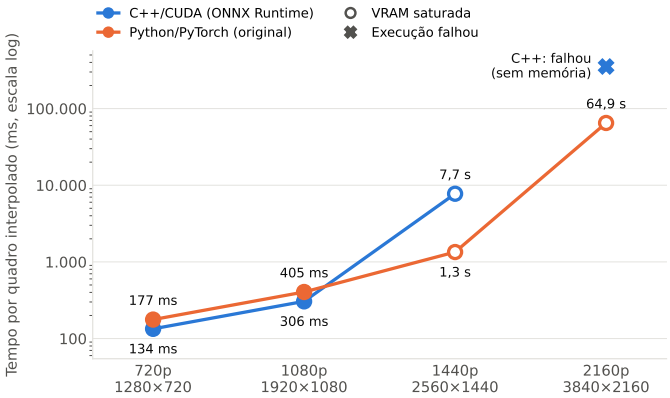
Configuração	ms	MiB
Anterior	307,3	6.646
Sem padrão de memória	305,7	4.743
Arena liberada a cada execução	363,2	4.743
Sem padrão, arena em pot. de 2	**306,2**	**4.885**

Desativar o padrão de memória: -29% de pico, sem custo de tempo. A combinação em destaque foi a adotada porque tem a menor memória sem custo de tempo nos blocos.

Meia precisão (FP16)

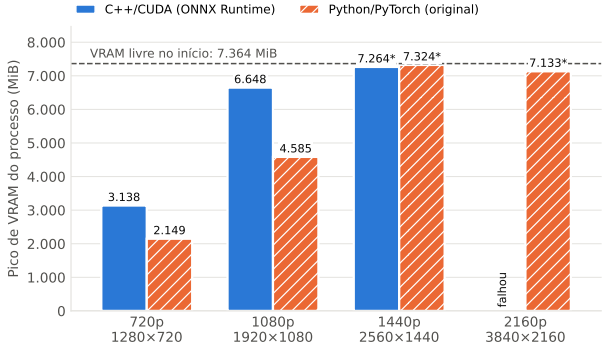
- Cada número ocupa 16 bits, em vez de 32.
- Rede e filtros em FP16: filtros de 1,6 GB para 0,8 GB em 1080p.
- Entradas, média e desvio, e acumulação da síntese seguem em FP32.
- Diferença de no máximo 1 nível, em cerca de 0,13% dos pixels.

v1: latência



- **Cerca de 24% mais rápido** em 720p e 1080p.
- 1080p: 305,5 ms contra 404,9 ms.
- H1 atendido, com $p < 0{,}001$.
- A partir de 1440p, a VRAM se esgota: 7,7 s contra 1,3 s.
- Em 2160p, o motor falha.

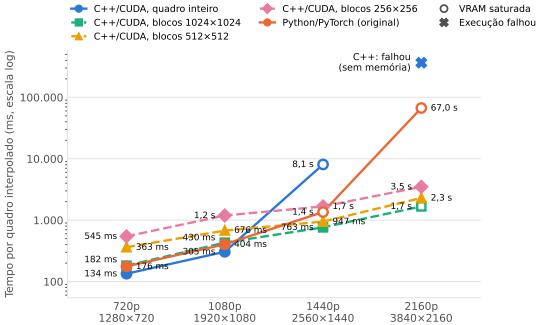
v1: consumo de VRAM

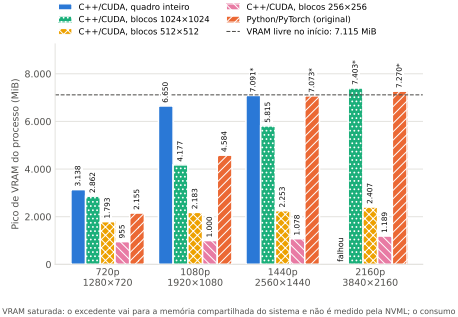


* VRAM saturada: o excedente vai para a memória compartilhada do sistema e não é medido pela NVML; o consumo real é maior que o valor mostrado.

- Contrário à hipótese: **cerca de 45% a mais** de VRAM.
- 1080p: 6.648 MiB contra 4.585 MiB.
- H2 não atendido.
- Trocar de linguagem, sozinho, não reduz a memória.

v2: memória limitada pelo bloco

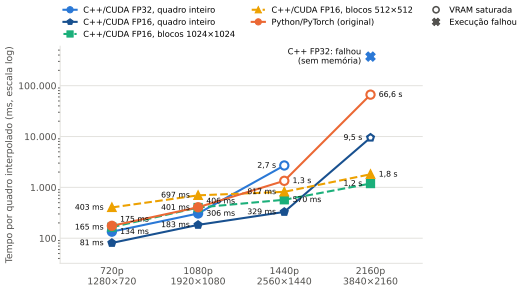




* VRAM saturada: o excedente vai para a memória compartilhada do sistema e não é medido pela NVML; o consumo real é maior que o valor mostrado.

- Blocos de 512: de 1.793 MiB (720p) a 2.407 MiB (2160p); em 2160p, 2,3 s contra 67,0 s.
- Custo: a rede processa de 1,15 a 2,9 vezes a área do quadro. Em 720p e 1080p, nenhuma configuração atende H1 e H2 ao mesmo tempo.

v3: FP16 com o quadro inteiro



Res.	Motor ms	FP16 MiB	Original ms	MiB
720p	80,8	1.554	175,3	2.149
1080p	183,0	3.049	406,3	4.582
1440p	329,0	4.800	1.345,8*	7.204*

* VRAM saturada.

- 1080p: -55% **de tempo e 67% da VRAM**.
- 1440p agora cabe, 4,1 vezes mais rápido.
- 2160p, FP16 com blocos de 1024: 1,19 s e 3.061 MiB, contra 66,6 s da original.

As quatro combinações

Res.	Python FP32 ms	Python FP32 MiB	Python FP16 ms	Python FP16 MiB	C++ FP32 ms	C++ FP32 MiB	C++ FP16 ms	C++ FP16 MiB
720p	176,0	2.146	132,3	1.837	133,6	2.940	80,2	1.555
1080p	405,5	4.585	303,9	3.801	306,0	4.888	183,0	3.057
1440p	1.343,7*	6.721*	543,6	6.583	2.635,3*	7.231*	329,0	4.771

* VRAM saturada. O C++ FP32 em 1440p atingiu 96,5% da capacidade, abaixo do limiar de 97%, e foi tratado como saturado pelo tempo.

- Comparar C++ em FP16 com Python em FP32 mistura dois efeitos.
- A original roda em FP16 com `torch.autocast`, sem alterar o código.
- Em FP16, o PyTorch alcança o motor em FP32: 303,9 ms contra 306,0 ms em 1080p.

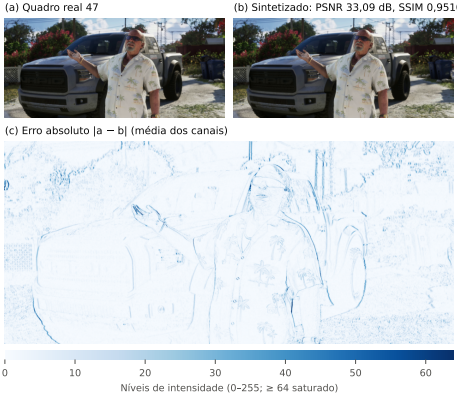
De onde vem cada ganho

Comparação	Latência 720p	Latência 1080p	VRAM (fração) 720p	VRAM (fração) 1080p
C++ × Python, em FP32	−24,1%	−24,6%	137%	107%
C++ × Python, em FP16	−39,4%	−39,8%	85%	80%
FP16 × FP32, no Python	−24,8%	−25,1%	86%	83%
FP16 × FP32, no C++	−40,0%	−40,2%	53%	63%
C++ FP16 × Python FP32	**−54,4%**	**−54,9%**	**72%**	**67%**

Todas as reduções de latência têm $p < 0{,}001$. VRAM: pico do primeiro termo como fração do pico do segundo.

- **Os dois fatores contam**, e nenhum explica sozinho os 55%.
- **Memória:** em FP32 o motor gasta mais; em FP16, menos. Parte disso vem de manter os filtros em FP16 na síntese, parcela que não foi isolada.

Fidelidade visual



Fonte: elaborado pelo autor, com quadros do trailer (ROCKSTAR GAMES, 2025).

	PSNR (dB)		SSIM	
	Midd.	Vimeo	Midd.	Vimeo
C++ FP32	37,22	34,46	0,9680	0,9590
C++ FP16	37,22	34,46	0,9679	0,9590
Python	37,22	34,46	0,9680	0,9590

- Vídeo em 1080p: 34,10 dB, SSIM 0,9538 e VMAF 77,4 nas duas implementações.
- H3 atendido em todas as versões.
- O erro se concentra nas bordas de objetos em movimento.
- A forma de execução não muda a qualidade.

Análise frente à hipótese (720p / 1080p)

Versão	Configuração	Latência	VRAM	H1 e H2
v1	FP32, quadro inteiro	−24,3% / −24,6%	+46% / +45%	só H1
v2	FP32, blocos 512	+106% / +67%	83% / 48%	só H2
v3	FP32, quadro inteiro	−23,3% / −24,6%	137% / 107%	só H1
v3	**FP16, quadro inteiro**	**−53,9% / −55,0%**	**72% / 67%**	**H1 e H2**
v3	FP16, blocos 1024	−6,0% / −1,3%	73% / 62%	só H2

Em relação à original em FP32. Na v1, a VRAM é a variação do pico; nas demais, a fração do pico da original.

- H3 foi atendido em todas as configurações.
- Contra a original também em FP16, o motor em FP16 ainda atende H1 e H2: −39,8% e 80% em 1080p.

Resposta à pergunta de pesquisa

Afirmativa, com uma ressalva sobre a origem de cada ganho

Em 1080p, o motor em FP16 levou 183,0 ms contra 406,3 ms, com 67% da VRAM e a mesma qualidade.

- **Latência:** vem da forma de execução, presente desde a v1, e é ampliada pela precisão numérica.
- **Memória:** não vem da troca de linguagem. Vem de decisões explícitas sobre o alocador, a precisão e, quando necessário, o processamento em blocos.
- O motor nativo torna essas decisões diretamente controláveis.

Contribuições

1. Constatação medida: trocar linguagem e *framework* reduz a latência, mas não reduz por si só a memória;
2. Separação do efeito da forma de execução e do efeito da precisão, pelas quatro combinações;
3. Identificação do padrão de memória do ONNX Runtime como principal fonte do excedente;
4. Quantificação do compromisso entre memória e latência no processamento em blocos;
5. Meia precisão com filtros em FP16 e acumulação em FP32, sem perda de fidelidade;
6. Protocolo de testes reprodutível, que considera o *Sysmem Fallback* do Windows;
7. Motor verificado contra a original e exportação ONNX corrigida.

Limitações e trabalhos futuros

Limitações

- Desempenho medido em um único trecho de 61 quadros, sem fixar os *clocks*.
- Em 2160p, só 3 medições e 4 quadros avaliados.
- Python em FP16 só pelo `autocast`.
- Vimeo-90K: 200 trios, de uma cópia pública.
- Condição de controle não executada; original medida só pelo tempo total.
- O limiar de 97% deixou de sinalizar uma saturação.

Trabalhos futuros

- Reduzir o custo dos blocos: tamanho automático, formato fixo e margens menores.
- Explicar por que o FP16 acelera mais o motor que o PyTorch, medindo por estágio.
- Comparar com TensorRT e quantização em 8 bits.
- Executar a condição de controle.
- Ampliar a base: mais vídeos, Vimeo-90K completo e repetições das baterias.



Obrigado!

Perguntas?

Pedro Henrique Gomes

Apoio: o que é FP16

	FP32	FP16
Bits por número	32	16
Dígitos de precisão, aproximados	7	3
Maior valor representável	$\approx 3{,}4 \times 10^{38}$	65.504
Menor passo perto de 1	$\approx 1{,}2 \times 10^{-7}$	$\approx 0{,}00098$

- Em FP16, $1 + 0{,}0004 = 1$: o termo pequeno some.
- Por isso, média, desvio padrão e acumulação da síntese seguem em FP32.
- A rede e os filtros ficam em FP16.

Apoio: ambiente experimental

Componente	**Especificação**
GPU	NVIDIA GeForce RTX 4060 Ti, 8 GB de VRAM
CPU e memória	Intel Core i5-12400F, 16 GB
Sistema	Windows 11 Home; *driver* 610.47 na 1ª bateria e 610.88 nas demais
Motor C++	C++17, CUDA Toolkit 13.3, MSVC (Visual Studio 2022), CMake 4.4
Inferência	ONNX Runtime 1.23.2 com *CUDA Execution Provider*
Vídeo	OpenCV 4.13
Original	Python 3.10, PyTorch 2.11 (CUDA 12.8)
Métricas	scikit-image 0.25 (PSNR e SSIM), FFmpeg 9.0 com *libvmaf*

Apoio: resultados da v1

Res.	Implementação	ms/quadro	FPS	VRAM (MiB)	PSNR (dB)	SSIM	VMAF
720p	C++/CUDA	133,9 ± 0,4	7,47	3.138	34,48	0,9622	85,16
720p	Python/PyTorch	176,8 ± 1,0	5,65	2.149	34,48	0,9622	85,16
1080p	C++/CUDA	305,5 ± 1,8	3,27	6.648	34,10	0,9538	77,42
1080p	Python/PyTorch	404,9 ± 2,2	2,47	4.585	34,10	0,9538	77,42
1440p	C++/CUDA	7.745,1 ± 214,5*	0,13	7.264*	34,04	0,9532	71,91
1440p	Python/PyTorch	1.344,5 ± 3,8*	0,74	7.324*	34,04	0,9532	71,91
2160p	C++/CUDA	falhou por falta de memória					
2160p	Python/PyTorch	64.900,5 ± 3.452,8*	0,02	7.133*	31,44	0,9323	50,33

* VRAM saturada: o excedente vai para a memória principal e não é contabilizado pela NVML.

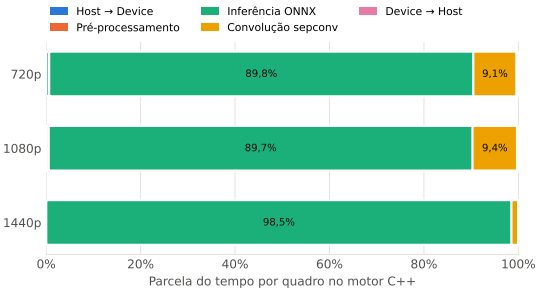
Apoio: resultados da v2

Res.	Implementação	ms/quadro	FPS	VRAM (MiB)
720p	C++/CUDA, quadro inteiro	134,1 ± 0,8	7,46	3.138
720p	C++/CUDA, blocos 1024	181,6 ± 8,0	5,51	2.862
720p	C++/CUDA, blocos 512	362,5 ± 19,2	2,76	1.793
720p	C++/CUDA, blocos 256	545,1 ± 24,4	1,83	955
720p	Python/PyTorch	176,2 ± 0,9	5,68	2.155
1080p	C++/CUDA, quadro inteiro	305,3 ± 1,8	3,28	6.650
1080p	C++/CUDA, blocos 1024	430,5 ± 5,4	2,32	4.177
1080p	C++/CUDA, blocos 512	676,0 ± 11,7	1,48	2.183
1080p	C++/CUDA, blocos 256	1.190,4 ± 29,3	0,84	1.000
1080p	Python/PyTorch	404,2 ± 1,8	2,47	4.584
1440p	C++/CUDA, quadro inteiro	8.074,8 ± 10,8*	0,12	7.091*
1440p	C++/CUDA, blocos 1024	762,7 ± 16,0	1,31	5.815
1440p	C++/CUDA, blocos 512	947,4 ± 10,3	1,06	2.253
1440p	C++/CUDA, blocos 256	1.673,7 ± 49,0	0,60	1.078
1440p	Python/PyTorch	1.354,7 ± 10,9*	0,74	7.073*
2160p	C++/CUDA, quadro inteiro	falhou por falta de memória		
2160p	C++/CUDA, blocos 1024	1.673,2 ± 5,0*	0,60	7.403*
2160p	C++/CUDA, blocos 512	2.296,4 ± 5,6	0,44	2.407
2160p	C++/CUDA, blocos 256	3.490,0 ± 5,6	0,29	1.189
2160p	Python/PyTorch	67.031,9 ± 2.189,8*	0,01	7.270*

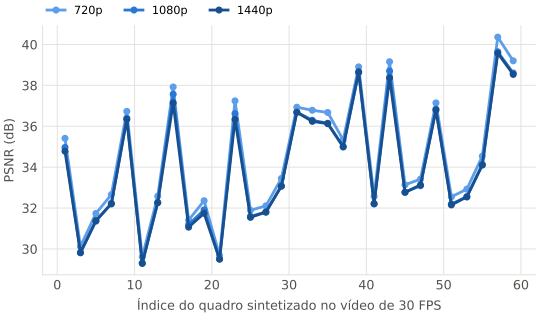
Apoio: resultados da v3

Res.	Implementação	ms/quadro	FPS	VRAM (MiB)
720p	C++/CUDA FP32, quadro inteiro	134,4 ± 0,9	7,44	2.944
720p	C++/CUDA FP16, quadro inteiro	80,8 ± 0,9	12,38	1.554
720p	C++/CUDA FP16, blocos 1024	164,7 ± 5,6	6,07	1.568
720p	C++/CUDA FP16, blocos 512	403,0 ± 11,5	2,48	893
720p	Python/PyTorch	175,3 ± 0,8	5,70	2.149
1080p	C++/CUDA FP32, quadro inteiro	306,2 ± 1,2	3,27	4.887
1080p	C++/CUDA FP16, quadro inteiro	183,0 ± 0,6	5,46	3.049
1080p	C++/CUDA FP16, blocos 1024	401,0 ± 24,9	2,49	2.855
1080p	C++/CUDA FP16, blocos 512	696,8 ± 37,3	1,44	920
1080p	Python/PyTorch	406,3 ± 1,7	2,46	4.582
1440p	C++/CUDA FP32, quadro inteiro	2.694,7 ± 9,7*	0,37	7.149*
1440p	C++/CUDA FP16, quadro inteiro	329,0 ± 0,8	3,04	4.800
1440p	C++/CUDA FP16, blocos 1024	570,3 ± 5,0	1,75	2.915
1440p	C++/CUDA FP16, blocos 512	816,7 ± 8,9	1,22	979
1440p	Python/PyTorch	1.345,8 ± 3,7*	0,74	7.204*
2160p	C++/CUDA FP32, quadro inteiro	falhou por falta de memória		
2160p	C++/CUDA FP16, quadro inteiro	9.502,7 ± 43,8*	0,11	7.370*
2160p	C++/CUDA FP16, blocos 1024	1.187,7 ± 2,2	0,84	3.061
2160p	C++/CUDA FP16, blocos 512	1.808,1 ± 12,7	0,55	1.127
2160p	Python/PyTorch	66.594,4 ± 1.768,6*	0,02	7.357*

Apoio: estágios e PSNR por quadro



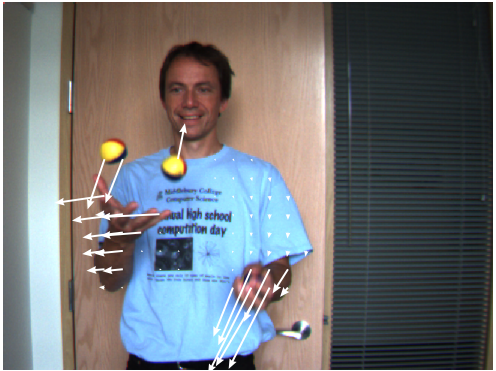
A inferência responde por cerca de 90% do tempo em 1080p; as transferências e o pré-processamento somam menos de 1%.



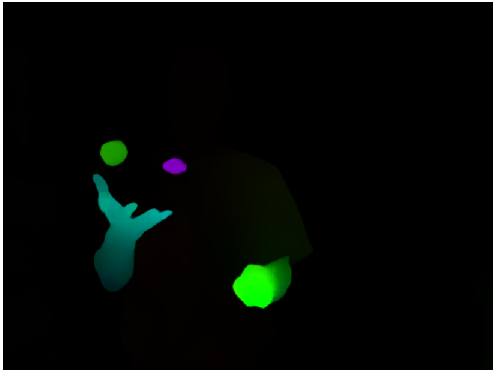
As curvas têm o mesmo formato nas três resoluções: a dificuldade depende do conteúdo de cada instante.

Apoio: fluxo óptico (RAFT)

(a) Vetores de deslocamento (u, v)



(b) Direção (cor) e magnitude (brilho) do fluxo



Fonte: elaborado pelo autor, com o RAFT da biblioteca torchvision e a sequência Beanbags (BAKER et al., 2011).

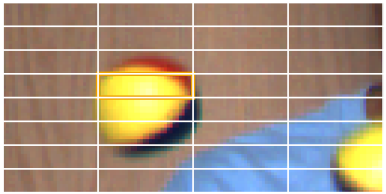
- Usado só nesta ilustração: não faz parte do motor nem das medições.

Apoio: execução paralela na GPU (SIMT)

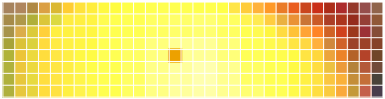
(a) Grade sobre a imagem



(b) Blocos de 32 × 8 threads



(c) Um bloco: cada thread, um pixel



Fonte: elaborado pelo autor, com a sequência Beanbags (BAKER et al., 2011).

- Uma grade de blocos cobre a imagem; cada bloco tem 32×8 *threads*.
- Cada *thread* calcula um pixel.

Apoio: por que usar mais de uma métrica

(a) Referência

(b) Borrada
PSNR 28,60 dB, SSIM 0,783

(c) Com ruído
PSNR 28,59 dB, SSIM 0,628



Fonte: elaborado pelo autor, com a sequência Beanbags (BAKER et al., 2011).

- Mesmo PSNR, cerca de 28,6 dB, e SSIM diferente: 0,783 e 0,628.
- O VMAF acrescenta uma avaliação perceptual, voltada a vídeo.

Apoio: escolha da margem (1080p)

Bloco	Margem	PSNR (dB)	Dif. máx.	ms/quadro
512	0	57,48	63	382
512	64	81,26	2	496
512	**96**	**98,36**	**1**	**570**
512	160	104,61	1	747
256	0	53,34	98	431
256	96	95,17	1	820

PSNR e diferença máxima em relação ao quadro inteiro, na reimplementação em PyTorch, em três pares de quadros.

Apoio: trecho do *kernel* de síntese

```
__device__ float accumulate_frame(const float* frame, const Filter* ver,
                                  const Filter* hor, ..., int x, int y, float acc[3])
{
    constexpr int R = K / 2;               // raio = 25
    float h[K]; int sx[K]; float hor_sum = 0.0f;

    for (int fx = 0; fx < K; ++fx) {       // le o filtro horizontal uma vez
        h[fx]  = to_float(hor[fx * filter_plane + filter_pixel]);
        sx[fx] = min(max(x + fx - R, 0), width - 1);   // borda replicada
        hor_sum += h[fx];
    }

    float ver_sum = 0.0f;
    for (int fy = 0; fy < K; ++fy) {       // para cada linha da vizinhanca
        const float v = to_float(ver[fy * filter_plane + filter_pixel]);
        ver_sum += v;
        const int sy = min(max(y + fy - R, 0), height - 1);
        const float* row = frame + (size_t)sy * width;

        float row_b = 0, row_g = 0, row_r = 0;
        for (int fx = 0; fx < K; ++fx) {   // convolucao horizontal da linha
            row_b += h[fx] * row[sx[fx]];
            row_g += h[fx] * row[plane + sx[fx]];
            // ... idem para o canal R; depois, acumula com o peso v
```

Os dois laços aninhados mostram as N^2 multiplicações por pixel: a separabilidade reduz a memória, não as contas.